Git y Otros Sistemas - Git como Cliente

Git como Cliente de Otros Sistemas

- Subversion: Git cuenta con un puente bidireccional que permite utilizar las características locales de control de versiones frente a un servidor de Subversion. Para que la integración funcione correctamente, es fundamental mantener un historial de desarrollo estrictamente lineal y evitar interacciones simultáneas con repositorios nativos de Git, ya que Subversion no procesa adecuadamente historiales divergentes o con fusiones.
- Mercurial: Mediante el uso de asistentes remotos, Git puede interactuar fluidamente con repositorios de Mercurial. Dado que ambos son sistemas distribuidos, el flujo de trabajo es muy similar al nativo, permitiendo mapear los marcadores del sistema externo como si fueran ramas locales, siempre y cuando el usuario se abstenga de reescribir historiales que ya han sido sincronizados.
- Perforce: Existen dos enfoques para la interoperabilidad. El primero es una integración del lado del servidor que expone áreas del depósito original como si fueran repositorios de Git con soporte completo para fusiones y ramificaciones. El segundo es un puente del lado del cliente que permite sincronizar y descargar versiones para trabajar localmente, requiriendo que el desarrollador alinee sus cambios en una línea de tiempo secuencial antes de publicarlos de vuelta en el servidor central.
- TFS (Team Foundation Server): Para interactuar con los repositorios centralizados de Microsoft, los desarrolladores pueden utilizar programas intermediarios basados en plataformas como Java o .NET. Estas soluciones extraen las versiones del servidor central hacia el entorno local, permitiendo al programador crear ramificaciones temporales o agrupar su trabajo, para finalmente subir el resultado empaquetado al servidor originario.

Migración Definitiva a Git

- Desde Subversion: La transición requiere descargar el proyecto completo mediante la herramienta de integración, aplicando un archivo de texto especializado para correlacionar las cuentas de usuario antiguas con identidades válidas para el nuevo sistema. Posteriormente, se debe ejecutar una limpieza manual para transformar las referencias remotas artificiales en etiquetas y ramas locales auténticas.
- Desde Mercurial: El proceso emplea secuencias de comandos de exportación rápida que traducen directamente los conjuntos de cambios al formato nativo del nuevo sistema. Durante la extracción, se corrigen las discrepancias en los nombres de los autores y se conservan de forma automática todas las etiquetas y marcadores originales.
- Desde Perforce: La exportación del código fuente se puede lograr utilizando la herramienta de sincronización del cliente para volcar toda la historia en un entorno local. Una vez finalizada la extracción, es necesario realizar una reescritura masiva del historial para purgar las marcas y metadatos de seguimiento que la herramienta de importación inyecta temporalmente en cada mensaje de confirmación.

- Desde TFS: Se recomienda utilizar los intermediarios de importación más robustos que posean la capacidad de comprender y replicar la estructura de ramas del servidor original. Tras aplicar un mapeo exhaustivo de las credenciales de los programadores, se debe limpiar la base de datos resultante para eliminar cualquier identificador residual del sistema anterior.
- Desde Sistemas Personalizados o Carpetas Físicas: Para herramientas sin soporte oficial o repositorios basados en simples copias de seguridad ordenadas por fechas, existe un mecanismo de importación rápida automatizada. El usuario puede programar rutinas simples que lean su estructura de directorios y emitan instrucciones de texto estandarizadas; el sistema asimila este texto de manera optimizada para ensamblar ramas, confirmaciones y archivos históricos desde cero